

Andrea Colombo - Algorithms for Massive Datasets

course project - A.A 2025/2026

Andrea Colombo

Abstract — This report contains the documentation related to the project submission for the *Algorithms for Massive Datasets* course.

Table of Contents

1	Introduction	2
1.1	Development Environment	2
2	Dataset Description	2
2.1	Preprocessing Techniques	2
2.2	Subsampling	2
3	System Configuration	2
4	Flajolet–Martin Algorithm	3
4.1	The Outlier Problem	3
4.2	Space/Time Complexity	3
4.3	Implementation Details	3
4.4	Experimental Results	3
5	AMS Algorithm	3
5.1	Space/Time Complexity	3
5.2	Implementation Details	3
5.3	Experimental Results	3
6	Conclusion	3
6.1	Future Work	3
7	Plagiarism and AI Usage Statement	3
	Bibliography	4

1 Introduction

1.1 Development Environment

This project was developed using *Google Colab* as the main computation environment. The Jupyter Notebook submitted alongside this project may require some modifications before being run in a local environment.

2 Dataset Description

The dataset used for this project is the **New York Times Articles & Comments (2020)** [1] dataset, freely available and licensed under the **CC-BY-NC-SA-4.0** license.

The dataset, once downloaded, has a size of approximately **6.15 GBs** presents itself in the form of multiple files:

- **nyt-articles-2020.csv**: Contains all the articles published in 2020 by the NYT.
- **nyt-comments-2020.csv**: this file contains all the comments relative to the articles found in *nyt-articles-2020.csv*.
- **nyt-comments-part0.csv .. nyt-comments-part9.csv**: these files simply contain the data found in *nyt-comments-2020.csv* split in 10 different partitions.
- **test.csv**: Not relevant for our use case
- **train.csv**: Not relevant for our use case

2.1 Preprocessing Techniques

During the preprocessing phase of the project we discarded all the columns that are not relevant for our use case, in particular:

- For the Flajolet-Martin portion of the project (Section 4), we discarded all the columns except the *UserID* and, since all the fields inside the schema are tagged as **nullable**, we excluded all the null entries to avoid using garbage data.

2.2 Subsampling

In order to ensure a reasonable execution time, we introduced a method that allows the user to load only a part of the dataset instead of the whole. This method can be tweaked by modifying the **SAMPLING_PROPORTION** (see: Section 3) variable inside the notebook provided alongside this document.

3 System Configuration

The python notebook submitted with this project can be configured with the following set of variables:

- **ENABLE_LOGGING**: enables additional prints during the notebook execution
- **SAMPLING_PROPORTION**: How much of the whole dataset we want to use for the current run, valid if in range (0, 1].
- **FM_NUM_HASHES**: Number of hash functions to use for the Flajolet-Martin implementation.

4 Flajolet–Martin Algorithm

First introduced in 1985 [2], the Flajolet-Martin algorithm is a probabilistic algorithm that aims at estimating the number of distinct elements through the use of hash functions.

The core idea is to leverage two very important properties of hash functions:

- **Determinism:** the hash function always maps the same value to the same result
- **Uniform Distribution:** the hashed values are uniformly distributed over the binary space

But why do we focus on the number of trailing zeros (tail length) of the hashed value?

The probability that a single hash value ends with n trailing zeros is $\frac{1}{2^n}$.

From this formula we can derive that:

- The probability of a single hash value not having n trailing zeros is $1 - \frac{1}{2^n} = 1 - 2^{-n}$
- The probability of **none** of k hash values having n trailing zeros is $(1 - \frac{1}{2^n})^k$. This can be approximated to $e^{-k2^{-n}}$ for very large numbers.

Therefore, if the maximum number of trailing zeros is $R_{\max}$, it implies that we have seen approximately $2^{R_{\max}}$ unique elements so far inside the stream.

4.1 The Outlier Problem

Using a single hash function has one big issue, since we are taking the maximum number of trailing zeros from a single source, we are susceptible to outliers.

The solution is simply using multiple hash functions and compute the median of their results. This has been verified to provide more accurate results but it still has one problem, it always results in estimates that are power of 2.

In order to fix this issue, we compute the average of the median estimations we just computed.

4.2 Space/Time Complexity

4.3 Implementation Details

4.4 Experimental Results

5 AMS Algorithm

5.1 Space/Time Complexity

5.2 Implementation Details

5.3 Experimental Results

6 Conclusion

6.1 Future Work

7 Plagiarism and AI Usage Statement

I declare that this material, which I now submit for assessment, is entirely my own work and has

not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.

Bibliography

- [1] B. Dornel, “New York Times Articles & Comments (2020).” 2021.
- [2] F. Philippe and M. G. Nigel, “Probabilistic Counting Algorithms for Database Applications,” *Journal of computer and system sciences* 31, 1985.